

Plan de Proyecto Software

A.1. Introducción

Todo sistema software, por modesto que sea su alcance, necesita algo más que código para llegar a buen puerto: necesita un plan. Un plan de proyecto traduce una idea inicial en un conjunto ordenado de decisiones (qué se va a construir y qué no, en cuánto tiempo, con qué recursos y bajo qué riesgos) y sirve a la vez de brújula durante el desarrollo y de vara de medir al terminar, cuando toca contrastar lo prometido con lo entregado. Como recuerda Sommerville (2016), la planificación es una de las actividades de gestión esenciales para entregar software de calidad dentro de plazo, de modo que dedicarle un anexo propio no es un formalismo, sino parte del trabajo de ingeniería.

Este anexo recoge la planificación del proyecto IUCE Reservas: su alcance, cómo se organizó el trabajo en el tiempo y el estudio de su viabilidad. Sirve como contrato de alcance y como referencia para evaluar las desviaciones entre lo planificado y lo entregado.

El desarrollo se ha conducido con Scrum, adaptado a un Trabajo de Fin de Grado individual: sprints de cadencia semanal, un Product Backlog vivo gestionado en Zube y revisiones periódicas con los tutores. El backlog está formado por 75 historias de usuario, agrupadas en doce épicas y repartidas en diez sprints. La estimación de cada historia se realizó de forma cualitativa, en puntos de historia sobre una escala de Fibonacci. Las cifras y figuras de este anexo proceden directamente de ese backlog.

A.1.1. Contexto y justificación

El IUCE gestionaba la reserva de sus aulas, laboratorios y salas de uso múltiple mediante un procedimiento manual basado en correo electrónico y una hoja de cálculo compartida. Ese flujo provocaba retrasos en las respuestas, errores de transcripción que derivaban en solapamientos, ausencia de notificaciones automáticas y una sobrecarga administrativa creciente. El proyecto se justifica por la oportunidad de digitalizar por completo ese proceso con una plataforma centralizada que reduzca la fricción operativa, dé autonomía al personal del instituto y aporte trazabilidad a las decisiones de asignación.

A.1.2. Marco metodológico

El desarrollo se ha conducido siguiendo un enfoque ágil. La filosofía ágil prioriza la entrega frecuente de software funcional, la adaptación al cambio y la colaboración continua por encima de una planificación cerrada de principio a fin; en lugar de definir todo el sistema por adelantado y construirlo en una única fase larga, el trabajo avanza en ciclos cortos que producen incrementos utilizables y revisables. Dentro de ese enfoque, el proyecto adopta Scrum, descrito por sus autores Schwaber y Sutherland (2020) como "un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos".

Scrum organiza el trabajo en iteraciones de duración fija llamadas sprints, al final de cada una de las cuales se obtiene un incremento del producto. El conjunto de todo lo que el sistema debe hacer se mantiene en el Product Backlog, una lista priorizada de necesidades. Esas necesidades se expresan como historias de usuario (descripciones breves de una funcionalidad desde el punto de vista de quien la usa, con el formato "como [rol], quiero [acción] para [beneficio]") y se agrupan temáticamente en épicas (conjuntos de historias afines a un área del sistema, como la autenticación o el motor de reservas).

Scrum define también tres responsabilidades dentro del equipo, tal y como describen Schwaber y Sutherland (2020); el Product Owner, que decide y prioriza qué aporta más valor y mantiene el Product Backlog; el Scrum Master, que vela por que el marco se aplique correctamente y elimina los obstáculos del equipo; y los desarrolladores, que construyen el incremento en cada sprint. En este Trabajo de Fin de Grado, al tratarse de un proyecto individual, el alumno asume de forma simultánea las tres responsabilidades, mientras que los tutores actúan como interlocutores que revisan el avance al cierre de cada bloque de trabajo, en un papel cercano al de cliente o parte interesada.

A.1.3. Alcance y delimitación

El sistema cubre el ciclo completo de la gestión de reservas de los espacios del IUCE. Ofrece un catálogo de espacios con filtros por capacidad, equipamiento y accesibilidad, y una autenticación institucional sin contraseña mediante magic link sobre las cuentas @usal.es. Su núcleo es un motor de reservas puntuales y recurrentes que valida los solapamientos, el aforo y los bloqueos administrativos, acompañado de un flujo de aprobación por parte del personal autorizado y de notificaciones tanto por correo como dentro de la propia aplicación. El usuario dispone además de una vista de calendario mensual con la ocupación de los espacios, y la administración cuenta con un panel de gestión que reúne espacios, bloqueos, usuarios, métricas, registro de auditoría, exportación a CSV y normas de uso.

Quedan fuera del alcance varias funcionalidades que, aun siendo razonables, no eran imprescindibles para resolver el problema planteado ni encajaban en el tiempo disponible. No se desarrolla una aplicación móvil nativa, ya que la interfaz web es responsive y se adapta a cualquier dispositivo sin necesidad de una app específica. Tampoco se implementa el inicio de sesión único institucional (SSO/SAML con idUSAL), que se plantea como línea de trabajo futura, ni la gestión multi-instituto o multi-organización (multi-tenant), porque el sistema se concibe para una única unidad. Se descarta asimismo la integración con calendarios externos como Outlook o Google Calendar. Por último, el sistema no permite reservar equipamiento de forma independiente (proyectores, portátiles y similares): el equipamiento no es un recurso reservable por sí mismo, si bien la ficha de cada espacio detalla con qué dotación cuenta, de modo que el usuario conoce de antemano los medios disponibles al elegir dónde reservar.

A.1.4. Entregables

El conjunto de entregables nos permite observar, por un lado, un producto software real y operativo y, por otro, la documentación que lo sustenta y lo hace mantenible. En el primer grupo se encuentra la aplicación web, desplegada en producción y accesible en reservas.iuce.usal.es; su código fuente, publicado en un repositorio público bajo licencia MIT; y las releases versionadas (v0.1.0 y v0.2.0), acompañadas de su changelog y de las evidencias de calidad correspondientes. El segundo grupo lo conforman la documentación técnica, recogida en los anexos de plan de proyecto, requisitos, diseño, programación, usuario y sostenibilidad, junto con la memoria del propio Trabajo de Fin de Grado. A estos se suma el Product Backlog gestionado en Zube, que se incluye como entregable propio porque recoge el proceso de planificación y mantiene la trazabilidad entre cada funcionalidad y la historia de usuario que la originó, más allá de constituir un mero registro interno. Así, los entregables cubren el alcance comprometido tanto en lo construido como en lo documentado.

A.1.5. Criterios y factores de éxito

Conviene distinguir entre ambas columnas. Los criterios de éxito son condiciones verificables que permiten comprobar de forma objetiva si el proyecto ha alcanzado su meta: el sistema en producción, el flujo completo operativo, la calidad asegurada y la trazabilidad. Los factores de éxito son las decisiones de proceso y de arquitectura que hicieron posible cumplir esos criterios. La *Figura 1* separa, por tanto, lo que se exige al resultado de las prácticas que lo hicieron alcanzable.

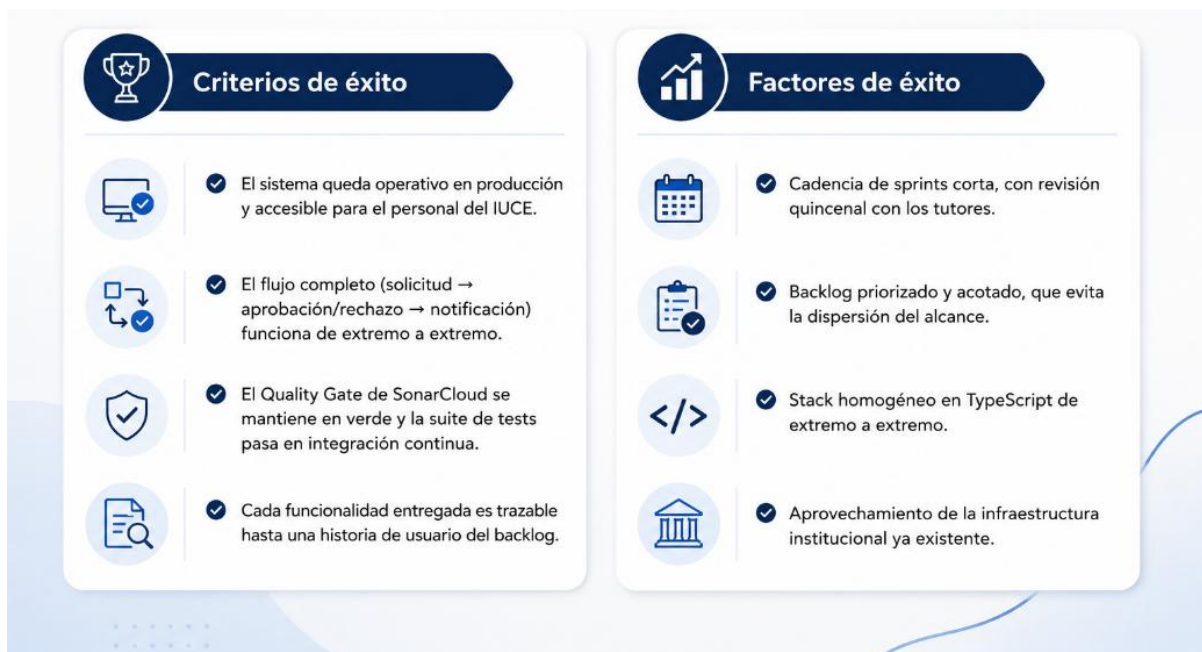


Figura 1. *Criterios y factores de éxito.*

A.1.6. Partes involucradas

En el proyecto intervienen varias partes con distintos roles y grados de implicación. La *Tabla 1* detalla cada parte y su responsabilidad, y la matriz RACI de la *Tabla 2* resume quién ejecuta, aprueba, es consultado o informado en cada gran actividad.

Tabla 1. *Partes involucradas en el proyecto.*

Parte	Rol	Interés / responsabilidad
Enrique González Gutiérrez	Autor / desarrollador	Análisis, diseño, implementación, pruebas y despliegue.
J. I. Santos y D. González (UBU)	Tutores del TFG	Dirección y revisión; actúan como Product Owners externos.
Dirección del IUCE	Cliente	Define las necesidades, valida el producto y ejerce de administrador.
CPD-USAL	Infraestructura	Servidor, dominio y soporte del despliegue en producción.
Personal del IUCE (PDI, PAS, investigadores)	Usuarios finales	Solicitan, consultan y cancelan reservas de espacios.

Más allá de su rol, las partes se distinguen por su nivel de interés en el sistema y su capacidad de influencia sobre él. La dirección del IUCE y los tutores combinan alto interés y alta influencia, por lo que concentran las decisiones de alcance y la validación; con ellos la estrategia es la implicación estrecha, mediante las revisiones quincenales. El personal del IUCE tiene alto interés pero baja influencia directa en las decisiones de diseño: su papel es proporcionar realimentación de uso, que se canaliza a través de la dirección. El CPD-USAL, finalmente, tiene baja implicación en el día a día funcional pero una influencia crítica en la fase de despliegue, lo que exige coordinación puntual pero imprescindible en ese momento del proyecto.

La asignación de responsabilidades sobre las grandes actividades del proyecto se resume en la matriz RACI, donde R indica quién ejecuta, A quién aprueba, C a quién se consulta e I a quién se informa. Dado el carácter individual del trabajo, el desarrollador concentra la ejecución, mientras que la aprobación y la consulta recaen en los tutores y la dirección del IUCE.

Tabla 2. Matriz RACI de responsabilidades del proyecto.

Actividad	Alumno	Tutores	Dirección IUCE	CPD-USAL
Análisis y especificación de requisitos	R	C	C	—
Diseño y arquitectura	R	C	I	—
Implementación y pruebas	R	I	—	—
Validación del producto	R	A	A	—
Despliegue en producción	R	I	I	C
Dirección y seguimiento del TFG	I	A/R	I	—

A.2. Planificación temporal

A.2.1. Estructura de descomposición del trabajo (EDT)

El trabajo se descompone en doce épicas, cada una de las cuales agrupa las historias de usuario de un área funcional del sistema. La *Figura 2* presenta la EDT hasta el nivel de épica, con el número de historias de cada una; el detalle de las historias se recoge en el catálogo de requisitos (*Anexo B*).

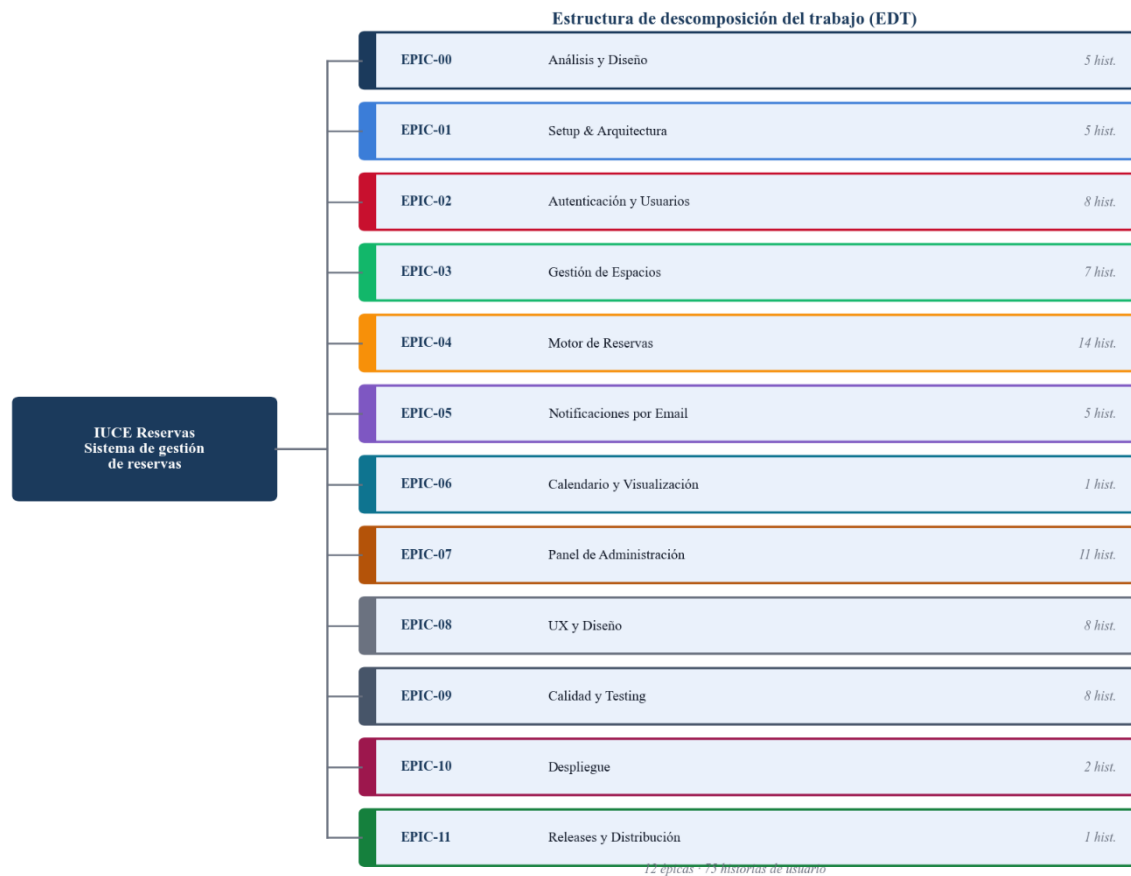


Figura 2. Estructura de descomposición del trabajo (EDT) - proyecto → épicas.

A.2.2. Cronograma de sprints

El proyecto se planificó en diez sprints de cadencia semanal, entre el 14 de abril y el 5 de junio de 2026, alineados con las tutorías quincenales (cada revisión cubría el avance de dos sprints). La *Figura 3* muestra el cronograma, con los hitos de publicación, y la *Tabla 3* detalla cada sprint.

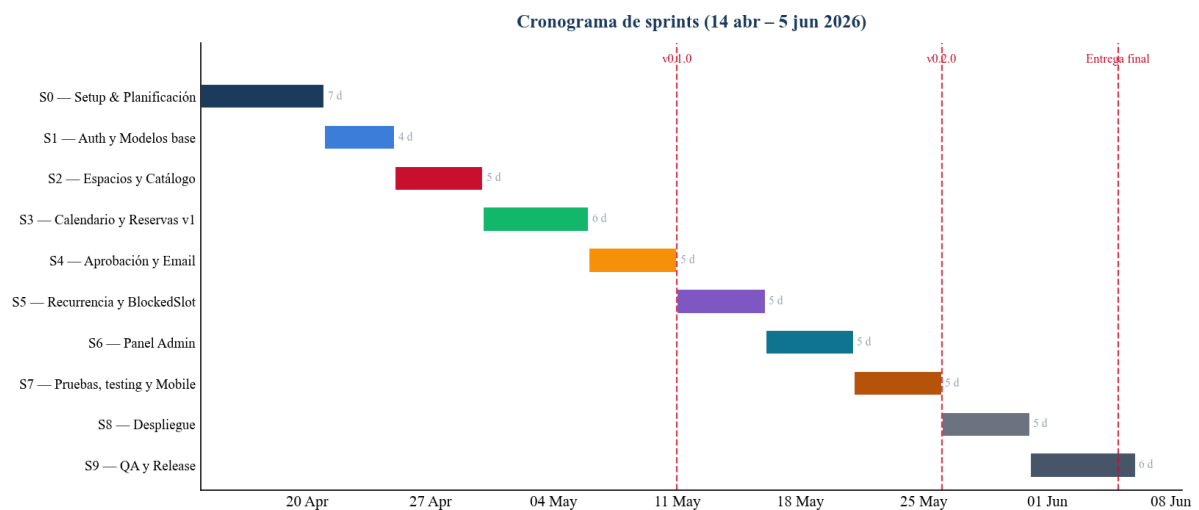


Figura 3. Cronograma de sprints (14 abr – 5 jun 2026), con los hitos de release.

La distribución semanal permitió mantener un ritmo de entrega constante y detectar desviaciones de forma temprana, ya que al cierre de cada sprint se disponía de incremento funcional revisable. El emparejamiento de cada tutoría con dos sprints consecutivos concentró la validación en hitos de mayor entidad, de modo que las decisiones de ajuste de alcance se tomaban sobre avances suficientemente consolidados.

A.2.3. Reparto del trabajo

El reparto por sprint (*Figura 4*) es regular (seis historias en la mayoría), con dos picos justificados. El Sprint 0 (diez historias) concentra el análisis y el diseño previos, incluidas las cinco historias de modelado. El Sprint 8 (doce) agrupa las funcionalidades finales antes del despliegue. Las tres historias “sin sprint” son tareas puntuales: un ajuste de acceso, la primera release y un enlace al catálogo.

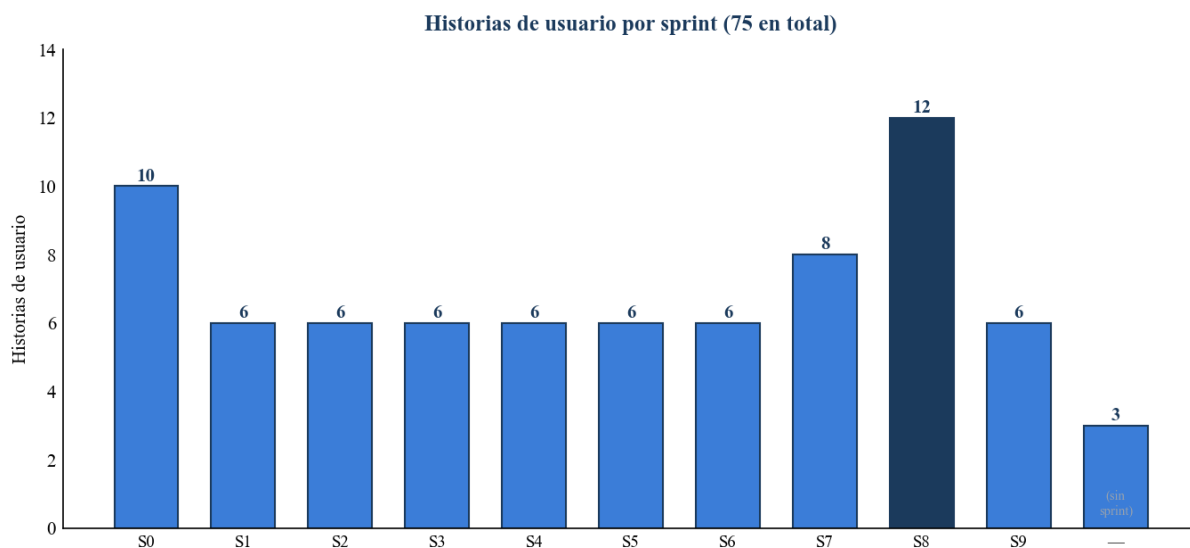


Figura 4. *Historias de usuario por sprint.*

Por épicas (*Figura 5*), el peso recae en el Motor de Reservas (14 historias), sección funcional del sistema, y en el Panel de Administración (11). La épica de Calidad y Testing (8) es transversal: acompaña a casi todos los sprints en lugar de concentrarse en uno.

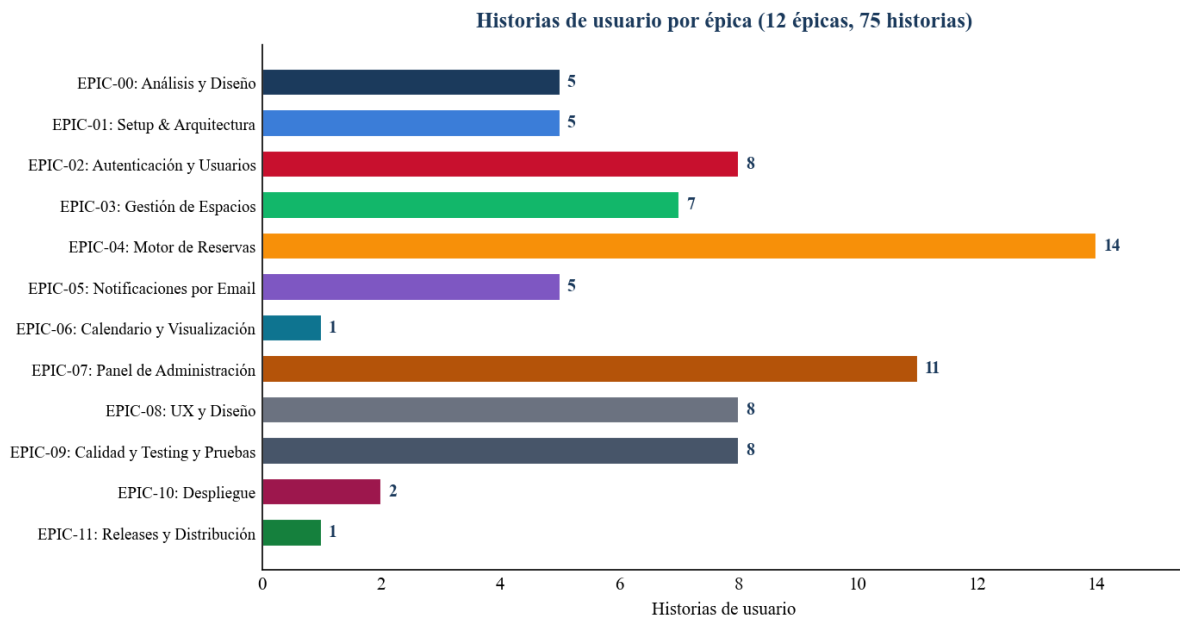


Figura 5. *Historias de usuario por épica (12 épicas).*

A.2.4. Avance y distribución del esfuerzo

El diagrama de avance acumulado (burnup, *Figura 6*) refleja una entrega sostenida (entre seis y ocho historias por semana) sin acumulación de trabajo al final, señal de un ritmo constante a lo largo de las ocho semanas.

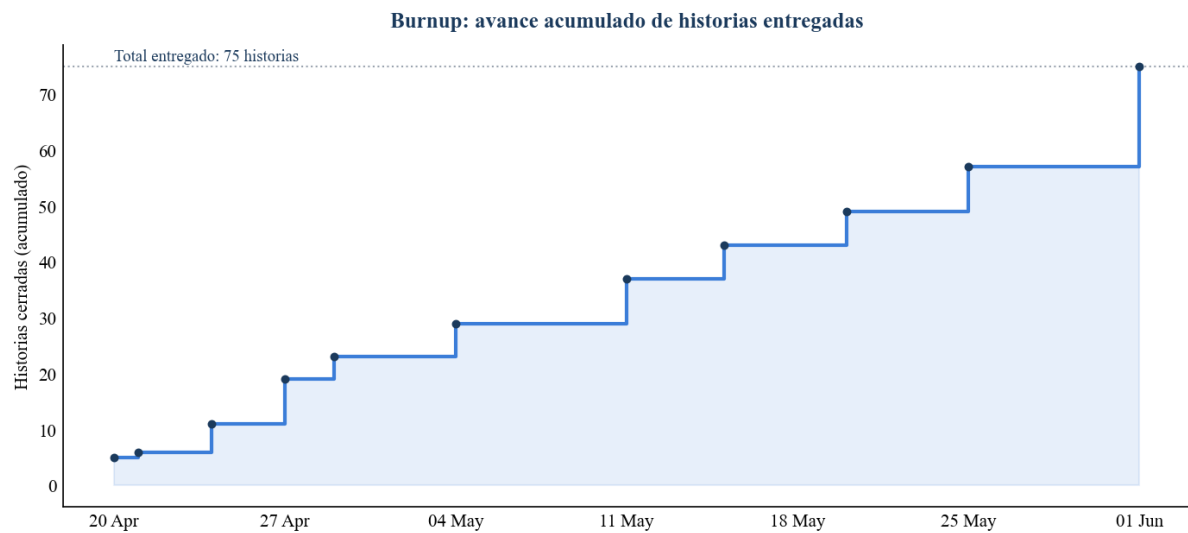


Figura 6. *Sprint Burnup - avance acumulado de historias entregadas.*

El mapa épica × sprint (*Figura 7*) nos permite observar la estructura del desarrollo: cada épica se concentra en su sprint temático (autenticación en el Sprint 1, espacios en el 2, reservas a partir del 3), mientras que el Motor de Reservas se extiende del Sprint 3 al 9 y la Calidad y Testing aparece en casi todos, reflejo de la verificación continua.

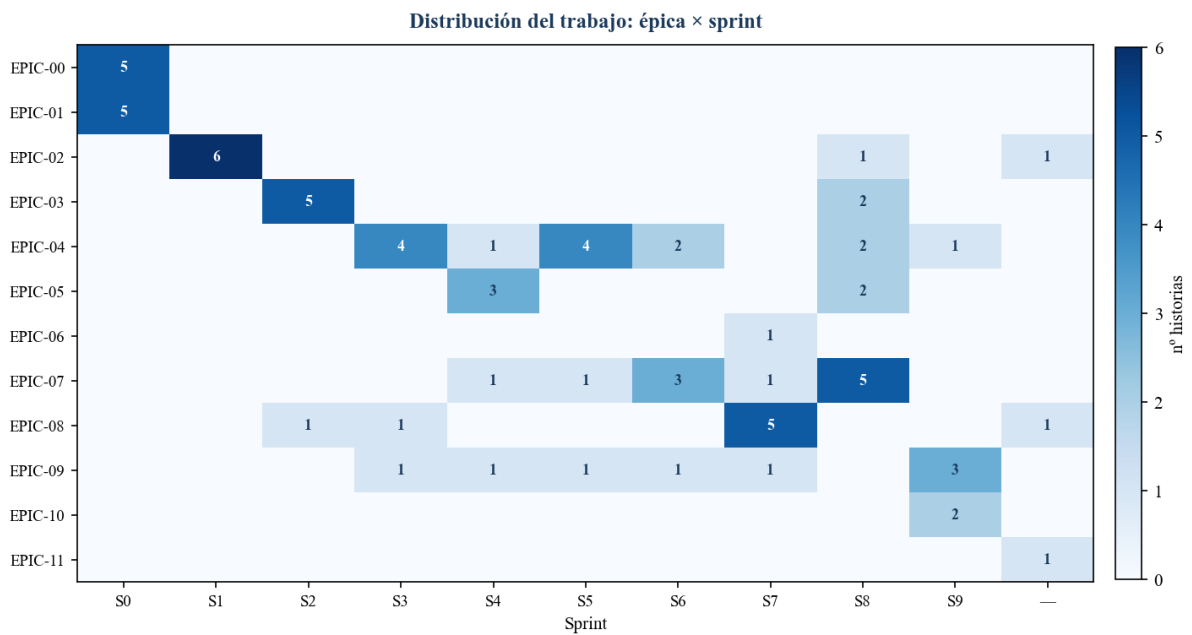


Figura 7. *Distribución del trabajo épica-sprint.*

A.2.5. Hitos y fechas objetivo

Más allá de la cadencia de sprints, el proyecto fijó los hitos de la *Tabla 4*, asociados a las releases y a la entrega final. A ellos se suman las revisiones quincenales con los tutores como puntos de control recurrentes.

Tabla 3. *Cronograma de sprints.*

Sprint	Fechas	Foco temático	Historias
S0	14–20 abr	Setup & Planificación	10
S1	21–24 abr	Auth y Modelos base	6
S2	25–29 abr	Espacios y Catálogo	6
S3	30 abr–5 may	Calendario y Reservas v1	6
S4	6–10 may	Aprobación y Email	6
S5	11–15 may	Recurrencia y BlockedSlot	6

Sprint	Fechas	Foco temático	Historias
S6	16–20 may	Panel Admin	6
S7	21–25 may	Pruebas, testing y Mobile	8
S8	26–30 may	Despliegue	12
S9	31 may–5 jun	QA y Release	6

A.2.6. Estimación del esfuerzo

El seguimiento del backlog se realizó en puntos de historia, no en horas, de modo que el esfuerzo dedicado no se registró de forma directa. Para dimensionar la viabilidad económica (sección A.3.1) se emplea una estimación de planificación: a partir de un presupuesto de referencia de 300 horas (el orden de magnitud de un Trabajo de Fin de Grado), el esfuerzo se reparte entre los sprints de forma proporcional al número de historias de cada uno, lo que equivale a unas cuatro horas por historia. La *Figura 8* muestra ese reparto. Conviene subrayar que es una estimación aproximada.

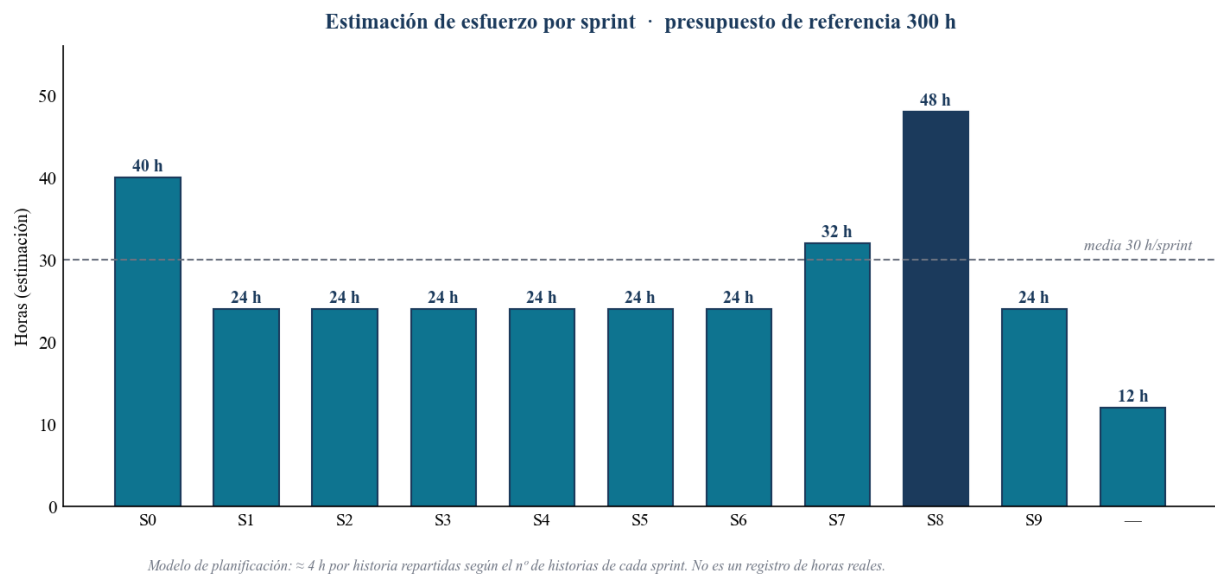


Figura 8. Estimación de esfuerzo por sprint (modelo de planificación 300 h).

A.3. Estudio de viabilidad

A.3.1. Viabilidad económica

El coste del proyecto se concentra casi por completo en el esfuerzo de desarrollo, ya que la infraestructura y los servicios se apoyan en software libre y en recursos institucionales de la USAL, sin coste imputable. La *Tabla 5* lo detalla.

Tabla 4. *Hitos y fechas objetivo.*

Hito	Fecha	Contenido
Inicio del desarrollo	14 abr 2026	Arranque del Sprint 0 (análisis y setup).
Release v0.1.0	11 may 2026	Cierre de Sprints 1-4: auth, catálogo, motor de reservas y aprobación.
Release v0.2.0	26 may 2026	Cierre de Sprints 5-7: recurrencia, bloqueos, panel admin y UX.
Entrega final	5 jun 2026	Cierre de Sprints 8-9: funcionalidades finales, despliegue y QA.

Coste de personal (partida dominante): aunque el proyecto no conlleva un desembolso económico real, el esfuerzo de desarrollo se valora como coste de desarrollo realizado por personal contratado, esto es, como el coste para la empresa de emplear a un programador júnior durante el tiempo dedicado, y no como una tarifa a precio de mercado. El coste para la empresa se descompone en el salario bruto proporcional a las horas trabajadas más las cotizaciones empresariales a la Seguridad Social. Como fuente salarial se toman las tablas de 2026 del XIX Convenio colectivo estatal de empresas de consultoría, tecnologías de la información y estudios de mercado y de la opinión pública, actualizadas mediante la Resolución de 13 de abril de 2026 de la Dirección General de Trabajo (BOE-A-2026-9024), por ser el convenio sectorial aplicable a la actividad de desarrollo de software. Para un perfil de programador de inicio de carrera se adopta el nivel E-1 del Área 3 (personal de informática), cuya retribución total para 2026 (salario base más plus convenio) asciende a 18.295,69 € anuales. Considerando la jornada anual de 1.800 horas que fija el convenio en su artículo 20, resulta un coste salarial de referencia de 10,16 €/h. Con el esfuerzo estimado de 300 horas del autor (véase la estimación de planificación de la sección A.2.6), el salario bruto proporcional asciende a 3.049,28 €.

Sobre ese salario bruto se aplican las cotizaciones empresariales a la Seguridad Social vigentes en 2026, conforme a la Orden PJC/297/2026, de 30 de marzo (BOE-A-2026-7296): contingencias comunes (23,60 %), desempleo en contrato indefinido (5,50 %), Fondo de Garantía Salarial (0,20 %), formación profesional (0,60 %), Mecanismo de Equidad Intergeneracional (0,75 %) y contingencias profesionales del sector (en torno al 1,30 %), lo que supone una aportación empresarial aproximada del 31,95 %. Estas cotizaciones ascienden a 974,25 €, de modo que el coste para la empresa se sitúa en 4.023,53 €. La suma de tecnologías y servicios (*Tabla 5*) es de 0 €, gracias al uso de software libre y de la infraestructura institucional de la USAL, por lo que la totalidad del coste del proyecto se concentra en el esfuerzo de desarrollo. En la práctica, el desembolso real del proyecto es nulo, lo que constituye precisamente uno de los principales argumentos de su viabilidad económica.

A.3.2. Viabilidad legal

La viabilidad legal de un proyecto software descansa sobre dos pilares diferenciados: la propiedad intelectual del código (tanto el propio como el de las dependencias incorporadas) y el cumplimiento de la normativa de protección de datos cuando el sistema trata información personal. Ambos aspectos se analizan a continuación, dado que condicionan tanto la posibilidad de distribuir y reutilizar el software como su despliegue en un entorno real.

En el plano de la propiedad intelectual, el proyecto se publica bajo la licencia MIT, una licencia de software libre de tipo permisivo que autoriza el uso, la modificación, la distribución y la sublicencia del código con la única condición de mantener el aviso de copyright y la exención de responsabilidad. Las licencias permisivas como MIT o Apache 2.0 se contraponen a las licencias de tipo copyleft, como la familia GPL, que obligan a que cualquier obra derivada se distribuya bajo la misma licencia, propagando así las condiciones de apertura al resto del desarrollo. La elección de una licencia permisiva resulta especialmente adecuada para un trabajo de orientación académica, pues maximiza la libertad de reutilización y la transferencia del sistema a otras unidades de la universidad sin imponer obligaciones a quienes lo adopten. Un aspecto crítico de la viabilidad legal es la compatibilidad entre la licencia del proyecto y las de sus dependencias de terceros: en este caso, todas las dependencias empleadas se distribuyen bajo licencias permisivas (MIT, Apache 2.0 e ISC), plenamente compatibles con la licencia MIT del proyecto y carentes de cláusulas copyleft que obligaran a liberar el conjunto del desarrollo bajo una licencia más restrictiva. No existe, por tanto, ninguna incompatibilidad de licencias ni restricción legal que comprometa la distribución del sistema.

La siguiente tabla resume el análisis de licencias realizado sobre las dependencias del proyecto y los componentes de la plataforma de ejecución. Todas las licencias identificadas son permisivas y compatibles con la licencia MIT bajo la que se distribuye el sistema, publicada en el fichero LICENSE del repositorio.

Componente	Versión	Licencia	Ámbito
Next.js	14.2.35	MIT	Producción
React / React DOM	18.3.1	MIT	Producción
NextAuth.js	5.0.0-beta.25	ISC	Producción
@auth/prisma-adapter	2.7.4	ISC	Producción
Prisma (cliente y CLI)	6.2.0	Apache 2.0	Producción / Desarrollo
Zod	3.24.1	MIT	Producción
Resend (SDK)	4.1.2	MIT	Producción
react-hot-toast	2.4.1	MIT	Producción
lucide-react	0.469.0	ISC	Producción
class-variance-authority	0.7.1	Apache 2.0	Producción
clsx	2.1.1	MIT	Producción
tailwind-merge	2.6.0	MIT	Producción
Tailwind CSS	3.4.17	MIT	Desarrollo (build)
TypeScript (compilador)	5.7.3	Apache 2.0	Desarrollo (build)
Vitest	2.1.8	MIT	Desarrollo (tests)
Testing Library (react, jest-dom, user-event)	15.0.7 / 6.9.1 / 14.6.1	MIT	Desarrollo (tests)
jsdom	25.0.1	MIT	Desarrollo (tests)
ESLint + eslint-config-next	8.57.0 / 14.2.0	MIT	Desarrollo
PostCSS / Autoprefixer	8.4.49 / 10.4.20	MIT	Desarrollo (build)
ts-node	10.9.2	MIT	Desarrollo
Definiciones de tipos (@types/*)	-	MIT	Desarrollo

Componente	Versión	Licencia	Ámbito
Node.js	20	MIT	Plataforma
PostgreSQL	16	Licencia PostgreSQL (permisiva, tipo BSD)	Plataforma
Docker Engine / Docker Compose	-	Apache 2.0	Plataforma
Apache HTTP Server	2.4	Apache 2.0	Plataforma

Resumen del análisis de licencias de las dependencias del proyecto y de la plataforma de ejecución.

Como se observa, las licencias MIT e ISC solo exigen conservar el aviso de copyright, y la licencia Apache 2.0 añade la obligación de mantener los avisos de atribución y la concesión expresa de patentes, condiciones todas ellas compatibles con la distribución del conjunto bajo MIT. Resend, como servicio externo de envío de correo, se rige por sus términos de uso; su SDK, que es el componente incorporado al código, se distribuye bajo MIT.

En materia de protección de datos, el sistema trata datos personales asociados a cuentas institucionales @usal.es, en concreto el nombre y el correo electrónico de los usuarios, lo que sitúa al proyecto en el ámbito de aplicación del Reglamento (UE) 2016/679, General de Protección de Datos (RGPD), y de la Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD), que adapta aquel al ordenamiento español. El artículo 5 del RGPD establece los principios que rigen todo tratamiento de datos personales, entre ellos la limitación de la finalidad y la minimización de datos. Respecto de esta última, el propio Reglamento dispone que los datos personales serán "adecuados, pertinentes y limitados a lo necesario en relación con los fines para los que son tratados". El diseño del sistema se ajusta a este mandato: solo se recogen los datos imprescindibles para gestionar las reservas, la finalidad del tratamiento queda acotada a ese cometido, no se produce cesión a terceros y la información se aloja en la infraestructura de la propia universidad.

La aplicación de estos principios no se limita a la fase de explotación, sino que, conforme al artículo 25 del RGPD, debe incorporarse desde el propio diseño del sistema (privacy by design) y por defecto (privacy by default). En esta línea, el sistema reduce su superficie de riesgo al prescindir por completo del almacenamiento de contraseñas: al sustituir la autenticación tradicional por un mecanismo de enlace mágico (magic link), se elimina toda una categoría de datos sensibles y los riesgos asociados a su custodia, como las filtraciones de credenciales o los ataques de reutilización de contraseñas. De este modo, las decisiones de arquitectura del proyecto satisfacen las exigencias normativas y, además, materializan el principio de protección de datos desde el diseño que el propio Reglamento erige en obligación del responsable del tratamiento.

A.3.3. Análisis de riesgos

Se identificaron los siguientes riesgos del desarrollo y del despliegue, con sus medidas de mitigación (*Tabla 6*).

Tabla 5. Coste de tecnologías y servicios.

Concepto	Licencia / plan	Coste imputable
Next.js, React, TypeScript, Prisma, Tailwind, NextAuth	Software libre	0 €
PostgreSQL 16	Software libre	0 €
SonarCloud	Gratuito (repositorios públicos)	0 €
GitHub	Gratuito	0 €
Zube	Plan académico / prueba	0 €
Resend (email transaccional)	Plan gratuito	0 €
Dominio reservas.iuce.usal.es	Institucional (USAL)	0 €
Servidor CPD-USAL + SSL (Let's Encrypt)	Institucional / gratuito	0 €

Tabla 6. Riesgos identificados y medidas de mitigación.

Riesgo	Prob.	Impacto	Mitigación
Dependencia del servicio de correo externo	Media	Medio	Envío asíncrono tras confirmar en la base de datos; usamos mock-Auth en desarrollo; proveedor sustituible.
Coordinación con el CPD-USAL para el despliegue	Media	Alto	Entorno reproducible con docker-compose.
Crecimiento del alcance (scope creep)	Media	Medio	Backlog priorizado; sprints de una semana; revisión quincenal con los tutores.
Desincronización del backlog entre Zube y GitHub	Media	Bajo	Zube como fuente de referencia del backlog; verificación periódica.

Riesgo	Prob.	Impacto	Mitigación
Baja adopción por los usuarios	Baja	Medio	Acceso sin contraseña ni instalación; interfaz sencilla; pruebas con usuarios.
Seguridad y protección de datos	Baja	Alto	Validación en servidor; autorización por rol; auditoría; sin contraseñas almacenadas.

A.3.4. Planes de calidad y comunicación

El plan de calidad parte de la premisa de que la calidad de un producto software no se inspecciona al final, sino que se construye de forma incremental a lo largo de todo el ciclo de desarrollo. Por ello se apoya en una estrategia de verificación automatizada que combina pruebas, análisis estático e integración continua, de manera que cada cambio introducido en el sistema queda sometido a un conjunto de comprobaciones objetivas antes de incorporarse a la base de código. Esta aproximación, que Sommerville (2016) sitúa entre las prácticas centrales de la ingeniería del software moderna al defender que las actividades de verificación y validación deben integrarse de forma continua a lo largo de todo el proceso de desarrollo, reduce el coste de detección de defectos al acercar su descubrimiento al momento en que se introducen y proporciona una red de seguridad que permite refactorizar y evolucionar el sistema con confianza.

El primer pilar es una batería de pruebas unitarias desarrolladas con Vitest y centradas en la lógica de negocio del sistema, esto es, en las validaciones de entrada, la máquina de estados de las reservas, el cálculo de la recurrencia y las utilidades de apoyo. Al concentrar el esfuerzo de prueba en el comportamiento crítico, buscamos que las reglas que gobiernan el dominio del problema queden cubiertas y verificadas de forma reproducible, evitando regresiones silenciosas cuando el código se modifica. El segundo pilar es el análisis estático con SonarCloud, que examina el código sin necesidad de ejecutarlo en busca de defectos potenciales, vulnerabilidades, duplicación y deuda técnica; su Quality Gate se configura como condición de integración, de modo que un cambio solo puede incorporarse si la métrica de calidad permanece en verde. El tercer pilar es la integración continua mediante GitHub Actions, que orquesta de forma automática y ante cada propuesta de cambio el análisis de estilo (lint), la comprobación de tipos de TypeScript, la ejecución completa de la suite de pruebas, y verifica además el build de producción únicamente en cada integración sobre la rama principal. Cierra el plan una convención de mensajes de commit que estructura el historial del repositorio de forma legible y trazable, vinculando cada cambio con la intención que lo motivó y facilitando tanto la revisión como la elaboración automática de los registros de versiones.

El plan de comunicación responde a la naturaleza del proyecto, en el que el autor desarrolla el sistema de forma autónoma bajo la supervisión de los tutores, que asumen el rol de Product Owners externos. En este contexto, la comunicación, por un lado, garantiza la alineación continua entre el trabajo realizado y los objetivos del proyecto y, por otro, deja constancia de las decisiones y del avance, de modo que en cualquier momento sea posible reconstruir el estado del desarrollo y los motivos que llevaron hasta él.

El eje principal en este sentido son las tutorías quincenales con los tutores, que actúan como punto de revisión del incremento entregado y de planificación del siguiente bloque de trabajo, en sintonía con las reuniones de revisión y retrospectiva propias de los marcos de trabajo ágiles. Estas reuniones permiten validar lo construido, recoger realimentación temprana y reajustar la priorización del backlog antes de afrontar el siguiente sprint. Entre revisiones, las releases versionadas en GitHub actúan también como hitos públicos del avance, acompañadas de su correspondiente changelog y de las evidencias de calidad asociadas, lo que ofrece una visión objetiva y fechada del estado del producto. Por último, el repositorio de código y el backlog gestionado en Zube actúan como registro compartido y revisable del progreso, enlazando cada funcionalidad entregada con la historia de usuario que la originó y consolidando así una documentación viva del proyecto que acompaña a su evolución.